

Projeto Final da Disciplina

Aplicação de metaheurísticas a um problema de otimização real

Lançamento: Semana 7 · Entrega final: Semana 16

Modalidade: grupos de até 5

Peso: 30% da nota final do componente curricular

1. Visão geral

O projeto final da INF0415 é a peça avaliativa onde você integra todo o ferramental da disciplina — busca local, busca populacional, otimização multiobjetivo, análise estatística, ética, XAI — em torno de um problema concreto de otimização. Não é um exercício de prova: é o lugar onde você se posiciona como alguém capaz de modelar um problema real, escolher metaheurísticas adequadas, executar experimentos rigorosos e comunicar os resultados de forma clara.

O projeto acompanha você ao longo das 10 semanas finais do semestre. A cada checkpoint, você adiciona uma camada nova de complexidade: começa com uma metaheurística single-objective, evolui para multiobjetivo, incorpora hibridização ou XAI, e termina com uma análise crítica completa. Nenhum aluno deve chegar à Semana 16 com tudo para fazer.

Objetivos de aprendizagem

Ao concluir o projeto final, você deverá ser capaz de:

1. Modelar um problema de otimização real, identificando variáveis de decisão, função(ões) objetivo, restrições e o tipo apropriado de representação computacional.
2. Selecionar e justificar a escolha de metaheurísticas em função das características do problema (combinatório vs. contínuo, single vs. multi-objective, com ou sem restrições).
3. Implementar pelo menos duas metaheurísticas distintas e configurá-las de forma reproduzível, com sementes, hiperparâmetros documentados e código versionado.
4. Executar um plano experimental estatisticamente válido, com múltiplas sementes, comparações pareadas e testes de significância.
5. Interpretar criticamente os resultados, discutindo trade-offs entre qualidade, robustez, custo computacional e adequação ao problema.
6. Comunicar o trabalho em formato técnico (relatório no estilo paper) e em apresentação oral.

2. Modalidade e formação de equipes

Tamanho da equipe

- Padrão: no máximo 5
- Trabalho individual é desencorajado, mas autorizado em situações de logística inviável (apresentar justificativa).

Divisão de trabalho dentro da equipe

A entrega final inclui um anexo de contribuições, descrevendo qual integrante foi responsável por cada parte. Avaliação majoritariamente conjunta, mas o professor pode atribuir notas individuais distintas se houver evidência de desbalanço significativo na contribuição.

Política de cooperação entre equipes

- Discussão livre de conceitos, ideias, dificuldades técnicas: encorajada.
- Compartilhamento de código entre equipes diferentes: não permitido. Cada equipe entrega seu próprio repositório, com seu próprio código.

- Reuso de código de bibliotecas (DEAP, pymoo, Optuna, scikit-learn): permitido e encorajado, com citação.
- Reuso de código pessoal de exercícios anteriores da própria disciplina: permitido.

3. Cronograma e checkpoints

O projeto não é entregue de uma vez. São cinco entregas parciais ao longo das semanas, cada uma com um produto concreto. As entregas parciais são curtas (1 a 3 páginas) e funcionam como bússola para garantir que você não acumule trabalho para a última semana.

Semana	Marco	Entrega	Peso
7	Lançamento	Apresentação da descrição do projeto em sala. Os alunos recebem este documento, a lista de problemas sugeridos e os materiais iniciais de estudo. Tempo livre para explorar.	—
9	Proposta	Documento de 1–2 páginas com: equipe, problema escolhido, justificativa, formulação preliminar (variáveis, objetivo(s), restrições), referências iniciais.	5%
11	Checkpoint 1	Modelagem completa em código + uma metaheurística single-objective rodando + baseline + análise inicial com 5 sementes. Notebook ou repositório executável.	15%
13	Checkpoint 2	Versão multiobjetivo (NSGA-II ou similar) + comparação com a versão single-objective + fronteira de Pareto inicial. Discussão de trade-offs.	20%
15	Pré-final	Versão integrada com pelo menos um diferencial (HPO, hibridização, XAI, warm-start, ética). Relatório técnico em primeira versão (mínimo 70% completo).	10%
16	Entrega final	Relatório técnico final + repositório de código + apresentação oral em sala (15 min + 5 min de perguntas).	50%

Política de atrasos

- Cada entrega parcial atrasada: –20% sobre o peso da entrega por dia útil de atraso, até no máximo 5 dias.
- A entrega final (Semana 16) não admite atraso — apresentação é em sala, no horário regular.
- Faltar a apresentação oral sem justificativa formal acarreta nota zero na parte de apresentação (10 pontos).

4. Requisitos mínimos

Estes seis requisitos são obrigatórios. Um projeto que falhe em qualquer um deles não pode receber nota máxima, independentemente da qualidade dos demais aspectos.

R1. Problema bem-formulado

O problema deve ter formulação matemática explícita: variáveis de decisão definidas com domínio, função(ões) objetivo expressas como expressão computável, restrições enumeradas. Vale para problemas combinatórios e contínuos.

R2. Pelo menos duas metaheurísticas distintas

Pelo menos uma deve ser populacional (AG, DE, PSO, ACO, NSGA-II/III). A outra pode ser de busca local (HC, SA, Tabu) ou outra populacional com paradigma diferente. Implementar duas variantes do mesmo método (ex.: AG com 2 crossovers diferentes) não satisfaz o requisito.

R3. Análise estatística válida

Mínimo de 10 sementes por configuração. Reportar média, desvio padrão, mediana, intervalo. Para comparações entre algoritmos, aplicar teste estatístico apropriado: Mann-Whitney U (ou Wilcoxon pareado quando aplicável) com correção de Bonferroni para múltiplas comparações. Reportar p-value e tamanho do efeito (não apenas significância).

R4. Pelo menos um baseline

Comparar com pelo menos uma referência: heurística construtiva específica do problema (vizinho mais próximo para TSP, EDD para scheduling), busca aleatória, ou solver exato em instâncias pequenas. Sem baseline, qualidade absoluta dos resultados é incompreensível.

R5. Visualização e comunicação dos resultados

Pelo menos 4 figuras de qualidade publicável: curva(s) de convergência com banda de desvio, boxplot ou violino comparativo, fronteira de Pareto (se MOO), e uma figura de análise específica do problema (mapa de rotas, escala de tempo, etc.). Eixos rotulados, legendas claras, sem capturas de tela.

R6. Reprodutibilidade

Repositório Git público (GitHub, GitLab) ou pasta zipada com: README claro, requirements.txt, sementes fixadas, instruções para reproduzir cada figura e tabela do relatório. Se não der pra rodar com 3 comandos, está incompleto.

5. Diferenciais e bonus

Cada projeto deve incorporar pelo menos um dos diferenciais abaixo (até a Semana 15). Projetos que incorporam múltiplos diferenciais bem executados recebem nota maior na rubrica de "profundidade".

D1. Hibridização (memetic algorithm)

Combinar uma metaheurística populacional com busca local. Exemplos: AG + 2-opt aplicado aos top-k indivíduos a cada N gerações; PSO + simulated annealing como passo de intensificação. Discutir trade-off entre custo extra e ganho em qualidade.

D2. Warm-start a partir de heurística clássica

Em vez de inicializar a população aleatoriamente, semear com soluções produzidas por heurística construtiva (ex.: vizinho mais próximo + perturbações). Comparar tempo até convergência e qualidade final entre cold-start e warm-start.

D3. Otimização de hiperparâmetros (HPO)

Aplicar Optuna (ou similar) para encontrar os melhores hiperparâmetros do método, dentro de um orçamento computacional fixo. Reportar a importância relativa dos hiperparâmetros e as configurações vencedoras. Tópico da Semana 12.

D4. XAI sobre o algoritmo

Aplicar IOHxplainer, EvoMapX ou ferramenta similar para entender o comportamento do algoritmo: quais hiperparâmetros importam mais? O que muda entre instâncias fáceis e difíceis? Vai além de "reportar resultado" e entra em "explicar o algoritmo". Tópico da Semana 13.

D5. LLM-driven evolution

Substituir o operador de crossover ou mutação por uma chamada a LLM (estilo FunSearch ou EoH). Implementação simplificada é aceitável: prompt que recebe dois indivíduos e retorna um filho "recombinado logicamente". Discussão crítica do custo vs. benefício. Tópico da Semana 14.

D6. Ética ou energia como objetivo

Incorporar uma dimensão ética ou de custo energético como objetivo formal da otimização — não como anexo. Exemplos: minimizar emissões além de custo; maximizar equidade no atendimento (geográfica, socioeconômica) além da eficiência; minimizar consumo do próprio algoritmo (FLOPs ou kWh). Tópico das Semanas 10 e 13.

D7. Conexão com problema real

Trabalhar com dataset público de impacto real (saúde pública, mobilidade urbana, agricultura, educação) ou em parceria com instituição (departamento, ONG, prefeitura). Documentar a conexão e discutir limitações de modelagem.

6. Problemas sugeridos

Você pode escolher um dos problemas abaixo ou propor um próprio (com aprovação do professor até a Semana 9). A lista cobre dificuldade variada e diferentes paradigmas. Para cada problema, sugerimos referências iniciais.

Problemas combinatórios

P1. VRP — Roteamento de veículos com múltiplos objetivos

- Minimizar distância total + número de veículos + tempo máximo de rota.
- Datasets clássicos: Solomon (VRPTW), Augerat. Disponíveis em CVRPLIB.
- Metaheurísticas naturais: AG com representação por permutação + delimitadores; NSGA-II; ACO.
- Diferenciais sugeridos: hibridização com 2-opt; warm-start com Clarke-Wright; ética via equidade de carga entre veículos.

P2. Job-Shop Scheduling

- Minimizar makespan e/ou atrasos ponderados.
- Instâncias clássicas: Taillard, Lawrence, Demirkol.
- Metaheurísticas: AG com representação por sequência de operações; NSGA-II para multi-objective.
- Diferenciais sugeridos: hibridização com Giffler-Thompson; HPO via Optuna.

P3. Knapsack multidimensional ou multiobjetivo

- Selecionar itens maximizando valor sob múltiplas restrições de capacidade, ou maximizando valor e minimizando peso simultaneamente.
- Datasets: MOOT, OR-Library, ZDT-knapsack.
- Metaheurísticas: AG binário; NSGA-II; SPEA2.
- Diferenciais sugeridos: XAI sobre quais itens entram com mais frequência; warm-start via greedy.

P4. Localização de facilidades em Goiás

- Dado um conjunto de municípios, decidir onde alocar k unidades (escolas, postos de saúde, antenas) para minimizar distância média ao serviço, maximizar cobertura populacional e respeitar restrição orçamentária.
- Dataset sugerido: Censo IBGE 2022 + dados do IMB (Instituto Mauro Borges).
- Metaheurísticas: AG binário; NSGA-II.
- Diferenciais sugeridos: ética via equidade espacial (índice de Gini geográfico); conexão com problema real.

P5. TSP com janelas de tempo (TSPTW)

- Versão restrita do TSP onde cada cidade tem janela de atendimento.
- Datasets: Dumas, Pesant, GendreauDumas. Disponíveis em sites acadêmicos.
- Metaheurísticas: AG com OX e penalização; SA com vizinhança 2-opt.

- Diferenciais sugeridos: hibridização AG + busca local; HPO.

P6. Roteiro turístico em Goiás

- Selecionar e ordenar atrações em uma viagem de N dias maximizando interesse total, minimizando deslocamento e respeitando orçamento. Variante prática do orienteering problem.
- Dataset: dados de pontos turísticos de Goiás (Goiandira, Pirenópolis, Chapada dos Veadeiros, Cidade de Goiás, Caldas Novas, etc.) com tempo de visita estimado e nota de interesse.
- Metaheurísticas: AG; NSGA-II.
- Diferenciais sugeridos: ética via diversidade regional (não concentrar em um polo); conexão real.

Problemas contínuos

P7. Otimização de portfólio financeiro

- Distribuir capital entre N ativos minimizando risco e maximizando retorno (problema clássico de Markowitz, em forma multiobjetivo).
- Dataset: yfinance (Python) com séries históricas de ações da B3 ou S&P 500.
- Metaheurísticas: DE; PSO; NSGA-II.
- Diferenciais sugeridos: ESG como terceiro objetivo; XAI sobre que tipos de portfólio o algoritmo prefere.

P8. NAS — Neural Architecture Search leve

- Dada uma tarefa de classificação simples (ex.: MNIST, Fashion-MNIST), buscar uma arquitetura de rede neural pequena que maximize acurácia, minimize parâmetros e minimize tempo de inferência.
- Espaço de busca pequeno: 2–4 camadas, kernel size, número de filtros.
- Metaheurísticas: AG codificado em vetor de hiperparâmetros; NSGA-II.
- Diferenciais sugeridos: ética via custo energético do treinamento; LLM-driven mutation propondo arquiteturas.

Problemas avançados

P9. Algorithm Discovery via LLM

- Aplicar a ideia central do FunSearch a um problema combinatório pequeno: usar um LLM como operador de mutação que reescreve uma heurística construtiva em código.
- Domínio sugerido: bin packing 1D, com função de score conhecida.
- Metaheurísticas: AG sobre programas, com chamadas a LLM como operador.
- Diferenciais: este projeto já vem com dois diferenciais embutidos (D5 + LLM-driven, D3 + HPO opcional). Indicado para equipes com background mais forte.

P10. Tema próprio (com aprovação)

Você pode propor um problema diferente, desde que:

- Tenha relevância clara (acadêmica, profissional ou social).

- Permita formulação como problema de otimização.
- Seja viável computacionalmente em laptop comum (sem GPU dedicada por horas).
- Seja apresentado por escrito até a Semana 9 e aprovado pelo professor.

7. Critérios de avaliação (rubrica)

Total: 100 pontos, distribuídos entre as entregas parciais (peso conforme cronograma) e a entrega final (50 pontos). A rubrica abaixo se aplica ao conjunto da entrega final + checkpoints anteriores.

Dimensão	Pontos	O que se avalia
Modelagem do problema	15	Clareza da formulação. Definição precisa de variáveis, objetivos, restrições. Adequação da representação computacional escolhida. Discussão das simplificações assumidas.
Implementação técnica	20	Qualidade do código (leitura, modularidade, documentação). Implementação correta das metaheurísticas. Reprodutibilidade. Versionamento.
Análise experimental	25	Plano experimental válido (sementes, número de execuções, controle de variáveis). Testes estatísticos apropriados. Comparação com baseline. Visualizações de qualidade.
Discussão crítica e profundidade	15	Capacidade de interpretar resultados além do reportar. Discussão de trade-offs, limitações e quando cada método é apropriado. Incorporação de diferenciais (D1–D7) com competência.
Relatório técnico	15	Estrutura adequada (paper-like). Escrita clara e correta. Citações apropriadas. Figuras e tabelas integradas ao texto. Cumprimento das normas de formatação.
Apresentação oral	10	Clareza na exposição. Capacidade de responder perguntas. Cumprimento do tempo. Qualidade dos slides.
Total	100	

Faixas qualitativas

- **90–100 (excelente):** trabalho com qualidade próxima de submissão a evento acadêmico nacional (ENIAC, BRACIS workshop). Múltiplos diferenciais bem executados, análise profunda, apresentação fluente.
- **75–89 (muito bom):** todos os requisitos cumpridos com competência, pelo menos um diferencial bem executado, análise consistente. Sem falhas significativas.
- **60–74 (suficiente):** requisitos atendidos no básico. Pode ter lacunas em análise estatística, profundidade da discussão ou qualidade do relatório.
- **abaixo de 60 (insuficiente):** falha em pelo menos um requisito mínimo, ou execução claramente apressada.

8. Estrutura do relatório técnico

Formato sugerido: estilo paper, duas colunas, template IEEE Conference (LaTeX) ou ACM. Extensão: 8–12 páginas (incluindo referências e figuras). Use template oficial; formatação correta importa.

Seções obrigatórias

7. Título e autores. Título descritivo (não "Trabalho final de heurísticas"). Autores com afiliação UFG.
8. Resumo (150–250 palavras). Estrutura: contexto + problema + método + principais resultados + conclusão.
9. Introdução (1 página). Motivação do problema, lacuna abordada, contribuições deste trabalho, organização do texto.
10. Formulação do problema (0,5–1 página). Definição matemática, instâncias usadas, métricas de avaliação.
11. Trabalhos relacionados (0,5–1 página). Não precisa ser exaustivo; cobrir os 5–8 trabalhos mais próximos do seu.
12. Metodologia (2–3 páginas). Algoritmos implementados (com pseudocódigo dos elementos não-triviais), representação, operadores, hiperparâmetros, design experimental.
13. Resultados (2–3 páginas). Tabelas e figuras com leitura clara. Cada figura tem texto que a discute (não apenas "a Figura X mostra os resultados").
14. Discussão (1 página). Interpretação dos resultados, trade-offs, limitações do método e do estudo, ameaças à validade.
15. Conclusões e trabalhos futuros (0,5 página).
16. Referências. ABNT ou IEEE, consistente. Mínimo 15 referências, das quais pelo menos 5 são artigos científicos (não tutoriais ou wikis).
17. Anexo de contribuições (1 parágrafo). Quem fez o quê dentro da equipe.

9. Estrutura da apresentação oral

15 minutos de apresentação + 5 minutos de perguntas. Exatamente. A organização da disciplina prevê todas as duplas no mesmo dia.

Estrutura sugerida (15 min)

- Problema e motivação (2 min)
- Formulação e instâncias (2 min)
- Metaheurísticas implementadas e principais decisões (3 min)
- Resultados — uma figura por aspecto (5 min)
- Discussão crítica e diferencial trabalhado (2 min)
- Conclusão (1 min)

Boas práticas

- Não ler slides. Ler é um sinal de que você não internalizou o trabalho.

- Não ter mais de 12 slides. Cada slide tem que merecer estar lá.
- Ter pelo menos uma figura discutida com profundidade. Mostrar que você entende o que aconteceu, não apenas o que rodou.
- Praticar com cronômetro pelo menos uma vez. 15 minutos passam mais rápido do que parece.
- Antecipar 2–3 perguntas que você teria se estivesse na audiência. Ter resposta na ponta da língua.

10. Materiais iniciais de estudo

Bibliografia geral

- EIBEN, A. E.; SMITH, J. E. *Introduction to Evolutionary Computing*. 2. ed. Berlin: Springer, 2015. (Texto-referência. Capítulos 3–6 cobrem AG, ES, EP, GP. Capítulo 12 cobre multi-objective.)
- TALBI, E.-G. *Metaheuristics: From Design to Implementation*. Hoboken: Wiley, 2009. (Visão abrangente, cobre busca local e populacional. Excelente para discussão de design choices.)
- DEB, K. *Multi-Objective Optimization Using Evolutionary Algorithms*. Chichester: Wiley, 2001. (Texto-referência específico para MOO. NSGA-II é descrito aqui.)
- LUKE, S. *Essentials of Metaheuristics*. 2. ed. Lulu, 2013. (Gratuito em cs.gmu.edu/~sean/book/metaheuristics.)

Artigos seminais

- DEB, K. et al. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Trans. Evolutionary Computation*, v. 6, n. 2, p. 182–197, 2002.
- STORN, R.; PRICE, K. Differential evolution — a simple and efficient heuristic for global optimization over continuous spaces. *J. Global Optimization*, v. 11, n. 4, p. 341–359, 1997.
- KENNEDY, J.; EBERHART, R. Particle swarm optimization. *Proc. ICNN*, 1995, p. 1942–1948.
- DORIGO, M.; STÜTZLE, T. *Ant colony optimization*. Cambridge: MIT Press, 2004. (Capítulos 1–3 estão em PDF disponível.)
- ROMERA-PAREDES, B. et al. Mathematical discoveries from program search with large language models. *Nature*, v. 625, p. 468–475, 2024. (FunSearch — base do diferencial D5.)

Frameworks e ferramentas

- **DEAP**: deap.readthedocs.io. AG, GP, ES. Familiar a vocês desde a Semana 7.
- **pymoo**: pymoo.org. Especializado em MOO. NSGA-II/III, SPEA2, MOEA/D, indicadores (hypervolume, IGD), benchmarks (ZDT, DTLZ). Recomendado fortemente para projetos com componente multiobjetivo.
- **Optuna**: optuna.readthedocs.io. HPO. Será introduzido na Semana 12.
- **MOOT**: Multi-Objective Optimization Test repository. Coleção de benchmarks padronizados. Será introduzido na Semana 15.
- **IOHxplainer / EvoMapX**: ferramentas de XAI sobre algoritmos evolutivos. Serão apresentadas na Semana 13.

Datasets e benchmarks

- TSPLIB (TSP, instâncias clássicas): elib.zib.de/pub/mp-testdata/tsp/tsplib
- CVRPLIB (VRP): vrp.atd-lab.inf.puc-rio.br

- OR-Library (knapsack, scheduling, e dezenas de outros): people.brunel.ac.uk/~mastjjb/jeb/info.html
- Instâncias Taillard (job-shop): mistic.heig-vd.ch/taillard/problemes.dir/ordonnancement.dir
- Censo IBGE 2022 e dados do Instituto Mauro Borges: imb.go.gov.br (para problemas geolocalizados em Goiás)

Templates

- LaTeX IEEE Conference: ieee.org/conferences/publishing/templates.html
- Template ACM: acm.org/publications/proceedings-template
- Para quem prefere Markdown + Pandoc: usar template eisvogel ou similar (disponíveis no GitHub).

11. Política de uso de IA generativa

Esta disciplina trata, entre outros temas, de evolução guiada por LLM. Ignorar a existência da IA generativa no contexto do trabalho seria incoerente. A política abaixo define o uso aceitável.

Permitido (com transparência)

- Usar LLM para entender conceitos, esclarecer dúvidas e revisar trechos de texto. Como você usaria uma colega para discutir.
- Usar LLM como assistente de programação (Copilot, Claude, ChatGPT) para escrever código auxiliar (visualizações, parsing de dataset, boilerplate). Documentar no relatório.
- Usar LLM como ferramenta dentro do próprio projeto (Diferencial D5).
- Usar LLM para ajustar a redação final do relatório, desde que o conteúdo seja seu.

Não permitido

- Submeter relatório onde o LLM gerou as ideias, a análise ou a discussão. O relatório precisa expressar o que VOCÊ entendeu sobre os resultados.
- Submeter código gerado por LLM sem entender. O professor pode pedir explicação de qualquer trecho durante a apresentação. Não saber explicar é evidência de uso indevido.
- Citar resultados ou referências geradas por LLM sem verificar. LLMs alucinam citações. Toda referência precisa existir e ter sido lida.

Declaração obrigatória

O relatório final deve incluir um parágrafo declarando os usos de IA generativa no projeto: quais ferramentas, em quais etapas, com qual extensão. Falta de declaração ou declaração incompatível com evidências constatadas é tratada como falta ética.

12. Dúvidas frequentes

Posso reusar código da Semana 7 (AG) ou de outras semanas?

Sim. O AG implementado na Sem.7, o Tabu da Sem.6, e o NSGA-II das Sem.12–13 são pontos de partida explícitos. Citar a reutilização não é "diminuir" o trabalho — é sinal de boa engenharia.

Posso mudar de problema depois da Semana 9?

Pode, mas não é recomendado. Mudanças entre Semanas 9 e 11 são aceitas com penalização leve no checkpoint. Após Sem.11, mudanças exigem justificativa muito forte.

Quanto tempo de computação isso vai exigir?

Para satisfazer R3 (10 sementes por configuração) com instâncias de tamanho moderado, você precisa pensar em código eficiente. Use seeds fixadas e teste com 2 sementes durante desenvolvimento; rode as 10 só para gerar a tabela final. Se a sua configuração final exige mais de 8h de máquina, considere reduzir o tamanho da instância.

Quão original o trabalho precisa ser?

Não precisa ser pesquisa de fronteira. Aplicar AG bem-implementado a um problema com formulação clara, com análise estatística válida, com comparação justa, é trabalho honesto. Os diferenciais (D1–D7) são onde o componente original aparece. Não tente "inventar uma metaheurística nova" — quase nunca dá certo em projeto de disciplina.

E se eu quiser publicar o trabalho depois?

Encorajado. ENIAC, BRACIS, SBPO recebem trabalhos de aluno. Combine com o professor para discussão de extensão. Trabalhos com nota acima de 90 frequentemente têm potencial de publicação após ajustes.

Posso continuar o projeto na Iniciação Científica ou TCC?

Sim, e isso é uma das melhores trajetórias. Conversar com o professor a partir da Sem.12 se houver interesse.

Boa sorte. Comece cedo. Pergunte muito.